

ISO SECURE

Plataforma de Cumplimiento ISO 27001

Etapa 2: Arquitectura y Diseño

Actividad Formativa 2 - 5 %

Agustín Peralta — Universidad de San Buenaventura

Entrega: 04 de Mayo de 2026

Resumen

Este documento presenta el diseño arquitectónico y la modelación de amenazas del proyecto ISO SECURE. Cubre tres ejes: (1) análisis de riesgos mediante metodología cualitativa Probabilidad $\times$ Impacto alineada a ISO 27005; (2) patrones de diseño aplicados en backend y frontend, justificados por los requisitos no funcionales identificados en la Etapa 1; (3) modelado de ataques híbrido STRIDE + PASTA, incluyendo árboles de ataque y catálogo priorizado de mitigaciones mapeadas a OWASP Top 10 y al Anexo A de ISO/IEC 27001:2022.

Índice

1. Introducción	3
2. Vista Arquitectónica General	3
2.1. Estilo arquitectónico	3
2.2. Componentes principales	3
2.3. Decisiones arquitectónicas clave (ADR)	3
3. Análisis de Riesgos	4
3.1. Metodología	4
3.2. Matriz Probabilidad $\times$ Impacto	4
3.3. Riesgos identificados por dominio ISO	5
3.4. Estrategia de tratamiento	6
4. Patrones de Diseño Aplicados	6
4.1. Backend (Python / FastAPI)	6
4.1.1. Layered Architecture	6
4.1.2. Dependency Injection (FastAPI Depends)	6
4.1.3. Factory Method para guards de rol	6
4.1.4. Strategy (detección de sector)	7
4.1.5. Repository (implícito) + Unit of Work (AsyncSession)	7

4.1.6. DTO / Schema (Pydantic v2)	7
4.2. Frontend (React 19)	7
4.2.1. Composition over inheritance	7
4.2.2. Container / Presentational	7
4.2.3. Adapter (api.js)	7
4.2.4. Guard de Vista (RBAC)	7
4.2.5. Lazy data loading	7
4.3. Patrones transversales	8
5. Modelado de Ataques	8
5.1. Metodología híbrida PASTA + STRIDE	8
5.2. Diagrama de Flujo de Datos (DFD nivel 1)	8
5.3. Análisis STRIDE	8
5.4. Árboles de ataque	10
5.4.1. Árbol 1: Brecha cross-tenant	10
5.4.2. Árbol 2: Falsificar métricas de cumplimiento	10
5.4.3. Árbol 3: Tomar control admin	10
5.5. Hoja de ruta de mitigaciones	10
6. Trazabilidad Requisitos → Diseño	10
7. Conclusiones	10

1. Introducción

La Etapa 2 traduce los requerimientos formulados en la Etapa 1 en una arquitectura concreta y verificable. El sistema ISO SECURE es una plataforma web multi-tenant para gestión del SGSI bajo ISO/IEC 27001:2022, con cuatro roles (admin, auditor, supervisor, analista) y nueve módulos operacionales.

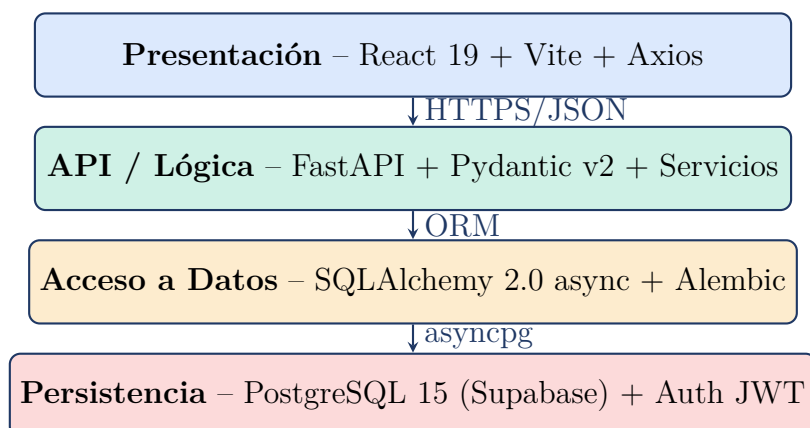
Premisas de diseño:

- **Seguridad desde el origen:** aplicar Privacy/Security by Design (ISO 27001:2022 cláusula 8.1).
- **Trazabilidad:** cada decisión arquitectónica debe responder a un requisito de la Etapa 1.
- **Modificabilidad:** preparada para crecer (módulos, sectores, integraciones n8n/Supabase).
- **Auditabilidad:** todo evento sensible debe quedar en bitácora (control A.8.15).

2. Vista Arquitectónica General

2.1. Estilo arquitectónico

ISO SECURE adopta una arquitectura **cliente-servidor por capas (Layered)** con separación estricta entre presentación, lógica y datos, comunicada por una API REST documentada con OpenAPI.



2.2. Componentes principales

2.3. Decisiones arquitectónicas clave (ADR)

ADR-01 — Stack FastAPI + Supabase. Adoptado por velocidad de prototipado, tipado fuerte vía Pydantic v2, soporte nativo async (alto throughput con bajo footprint) y Auth JWT gestionada por Supabase (delegamos rotación de claves, MFA y políticas de contraseña).

ADR-02 — Monolito modular en lugar de microservicios. Volumen de tráfico esperado (<200 usuarios concurrentes en piloto), equipo reducido y necesidad de iterar rápido. Si crece, los routers ya están aislados por dominio y permiten extracción posterior.

Componente	Responsabilidad	Tecnología
SPA Frontend	Vistas dashboard, incidents, controls, risk, etc.	React 19 + Vite
API Gateway (Nginx)	Reverse proxy + TLS termination	Nginx alpine
API REST	39 endpoints distribuidos en 9 routers	FastAPI 0.111+
Servicios	Lógica de negocio por dominio	Python 3.13
ORM	Maapeo objeto-relacional async	SQLAlchemy 2.0
Auth Provider	Emisión/validación de JWT, refresh tokens	Supabase Auth
Base de Datos	9 tablas, multi-tenant por empresa_id	PostgreSQL 15
Migraciones	Esquema versionado declarativo	Alembic
Orquestador	Webhooks de reportes y alertas	n8n self-hosted

ADR-03 — Multi-tenant lógico (empresa_id FK). Comparte schema entre tenants para simplificar mantenimiento y reducir costos. Acepta el riesgo de fuga cross-tenant a cambio de aplicar control estricto en cada query (helper `_empresa_filter`) y pruebas de seguridad obligatorias.

ADR-04 — JWT validado contra Supabase. En lugar de validar localmente la firma, validamos cada request contra `/auth/v1/user`. Trade-off: latencia adicional vs. revocación inmediata de sesiones comprometidas.

3. Análisis de Riesgos

3.1. Metodología

Se aplica análisis de riesgos cualitativo conforme a **ISO/IEC 27005:2022**, complementado por la matriz de impacto del Anexo A. Cada riesgo se evalúa por:

$$\text{Nivel de Riesgo} = \text{Probabilidad} \times \text{Impacto}$$

Las escalas son:

Nivel	Valor	Descripción
Muy Baja / Insignificante	1	Improbable / sin impacto material
Baja / Menor	2	Posible / pérdida limitada
Media / Moderada	3	Probable en 12 meses / interrupción parcial
Alta / Mayor	4	Esperable / pérdida significativa
Muy Alta / Catastrófica	5	Recurrente / pérdida total o legal

3.2. Matriz Probabilidad × Impacto

Prob.Imp.	1	2	3	4	5
5	5	10	15	20	25
4	4	8	12	16	20
3	3	6	9	12	15
2	2	4	6	8	10
1	1	2	3	4	5

Convención de colores: Verde (1-4): aceptable; Amarillo (5-12): tolerable con monitoreo; Rojo (≥ 15): requiere mitigación inmediata.

3.3. Riesgos identificados por dominio ISO

ID	Riesgo	P	I	R	Dominio ISO
R-01	Fuga de datos cross-tenant por bypass del filtro <code>empresa_id</code>	3	5	15	A.5.15, A.8.3
R-02	Token JWT robado por XSS en frontend	2	5	10	A.5.17, A.8.5
R-03	Inyección SQL vía parámetros no sanitizados	1	5	5	A.8.28
R-04	Denegación de servicio por exceso de requests (no rate limit)	4	3	12	A.8.6
R-05	Escalamiento de privilegios por endpoint mal protegido	2	5	10	A.5.15, A.8.2
R-06	Pérdida de datos por ausencia de respaldos validados	2	5	10	A.8.13
R-07	Repudio: ausencia de bitácora de cambios sensibles	3	4	12	A.8.15
R-08	Almacenamiento de secretos en variables de entorno sin gestión	3	4	12	A.8.24
R-09	Dependencias con vulnerabilidades conocidas (CVE)	4	3	12	A.8.8
R-10	Ataques CSRF a endpoints autenticados	2	3	6	A.8.26
R-11	Subida de archivos maliciosos (futuras integraciones)	2	4	8	A.8.7
R-12	Fallo en la rotación de claves JWT de Supabase	1	5	5	A.5.17
R-13	Ingeniería social a usuarios privilegiados	3	4	12	A.6.3
R-14	Logs con información sensible (tokens, PII)	3	3	9	A.8.11, A.8.15
R-15	Indisponibilidad del proveedor Supabase	1	4	4	A.5.30

Resumen del perfil de riesgo:

- Riesgos críticos (≥ 15): **1** (R-01)
- Riesgos altos (9-12): **7**

- Riesgos moderados (5-8): **6**
- Riesgos bajos (1-4): **1**

3.4. Estrategia de tratamiento

Conforme a ISO 27005, cada riesgo se trata con una de las siguientes opciones:

Estrategia	Aplicación en ISO SECURE
Mitigar	R-01, R-02, R-03, R-04, R-05, R-07, R-08, R-09 (la mayoría)
Transferir	R-15 (cobertura por SLA del proveedor Supabase)
Aceptar	R-12 (bajo, controlado por Supabase)
Evitar	R-11 (no se permite subida de archivos hasta integrar antivirus)

4. Patrones de Diseño Aplicados

4.1. Backend (Python / FastAPI)

4.1.1. Layered Architecture

Separación estricta entre **Router** (HTTP), **Service** (negocio), **Repository implícito vía SQLAlchemy** (datos) y **Schema** (contratos). El router nunca toca la base de datos directamente.

Por qué: reduce acoplamiento, facilita testing unitario por capa y permite cambiar el ORM o el motor HTTP con bajo costo.

4.1.2. Dependency Injection (FastAPI Depends)

Cada router declara dependencias explícitas: sesión async, usuario autenticado, filtros de rol.

```
@router.get("/incidents")
async def list_incidents(
    db: AsyncSession = Depends(get_db),
    user: UserProfile = Depends(get_current_user)
):
    ...
```

Por qué: elimina globals, hace explícitas las dependencias y habilita sobreescritura en pruebas (`app.dependency_overrides`).

4.1.3. Factory Method para guards de rol

`require_role(["admin", "auditor"])` retorna una dependency parametrizada.

Por qué: elimina código repetido de chequeo de roles en cada endpoint y centraliza la política de autorización.

4.1.4. Strategy (detección de sector)

`_detect_sector()` aplica reglas por palabras clave para asignar el catálogo de controles ISO correspondiente al sector (finanzas, salud, tecnología, etc.).

Por qué: permite agregar nuevos sectores sin tocar el flujo de creación de empresa.

4.1.5. Repository (implícito) + Unit of Work (AsyncSession)

Cada `AsyncSession` encapsula una transacción. Los servicios reciben la sesión y operan sobre ella; el commit/rollback es responsabilidad del scope superior.

4.1.6. DTO / Schema (Pydantic v2)

Separación entre modelos ORM (`models/`) y contratos de API (`schemas/`). Evita exponer columnas internas y permite validación estricta de entrada.

4.2. Frontend (React 19)

4.2.1. Composition over inheritance

Componentes pequeños y componibles (no jerarquías de clases). Vistas declarativas con props tipadas implícitamente.

4.2.2. Container / Presentational

`App.jsx` actúa como contenedor (estado global, fetch, routing manual por vista); los componentes hijos son presentacionales.

Deuda técnica reconocida: actualmente `App.jsx` pesa ~2100 líneas. Plan de refactor a contextos + componentes de vista cuando exceda 3000 líneas.

4.2.3. Adapter (api.js)

Capa de adaptación sobre Axios: interceptor de token, manejo unificado de errores y exports por dominio (`incidentApi`, `controlApi`, etc.).

Por qué: aísla la lógica de transporte HTTP; si cambiamos Axios por fetch nativo, sólo se modifica este archivo.

4.2.4. Guard de Vista (RBAC)

Constante `VIEW_ROLES` mapea vista → roles permitidos; componente `AccessDenied` se renderiza si el usuario actual no cumple.

4.2.5. Lazy data loading

Cada vista trae sólo los datos necesarios al montarse (`useEffect`), evitando cargas innecesarias.

4.3. Patrones transversales

- **Configuration via Environment:** 12-Factor App; secretos fuera del código fuente.
- **Migration as Code:** Alembic + script `migrate.py` para seeding determinístico.
- **Healthcheck endpoint:** `/health` para liveness/readiness en Docker.
- **Versionado de API:** prefijo `/api/v1` para permitir breaking changes futuros.

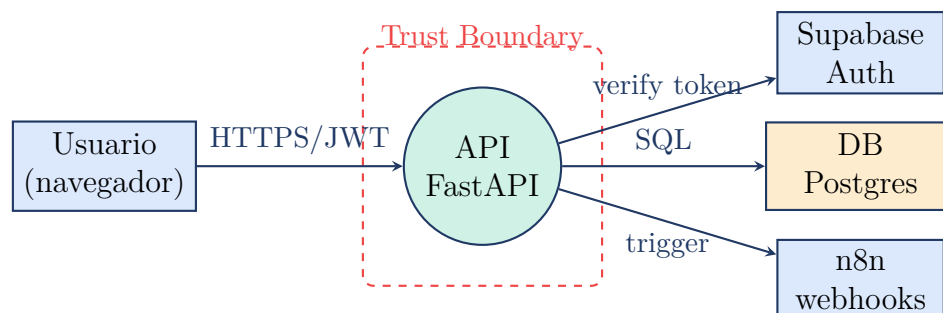
5. Modelado de Ataques

5.1. Metodología híbrida PASTA + STRIDE

Se adopta **PASTA** (Process for Attack Simulation and Threat Analysis) como marco de proceso, integrando **STRIDE** como taxonomía de amenazas en la Etapa IV. Comparativa de metodologías evaluadas:

Metodología	Enfoque	Decisión
PASTA	Riesgo, alineada a negocio, 7 etapas	Adoptada
STRIDE	Taxonomía por categoría de amenaza	Integrada en PASTA IV
VAST	Visual, ágil, para DevOps	Considerada para automatización futura
OCTAVE	Operacional, corporativo	Descartada (overkill para tamaño actual)
DREAD	Scoring numérico de impacto	Descartada (subjetividad alta)
LINDDUN	Privacidad (GDPR)	Considerada para módulo de PII

5.2. Diagrama de Flujo de Datos (DFD nivel 1)



Las **trust boundaries** son los puntos donde cambia el nivel de confianza. Toda entrada que cruza una boundary debe ser validada, autenticada y autorizada.

5.3. Análisis STRIDE

ID	Categoría	Amenaza	Mitigación
S-01	Spoofing	Falsificación de JWT	Validación contra Supabase /auth/v1/user
S-02	Spoofing	Login con credenciales robadas	MFA opcional + monitoreo de IPs anómalas
S-03	Spoofing	Reutilización de session tokens	Tokens de corta vida + refresh tokens rotados
S-04	Spoofing	Falsificar empresa_id en payload	Inferir empresa_id del JWT, no del request
T-01	Tampering	Modificar controles vía API directa	Validación de rol + filtro empresa_id
T-02	Tampering	SQL Injection	ORM parametrizado + Pydantic validation
T-03	Tampering	Tamper de respuestas en tránsito	TLS 1.3 obligatorio + HSTS
T-04	Tampering	Inyección de datos en upload (futuro)	Validación MIME + antivirus
R-01	Repudio	Negar haber editado un control	Bitácora con user_id + timestamp
R-02	Repudio	Negar haber bajado un reporte	Log de exports en tabla audit_log
R-03	Repudio	Borrado de logs por insider	Logs append-only + envío a sink externo
I-01	Info Disclosure	Cross-tenant leakage	Helper __empresa_filter en cada query
I-02	Info Disclosure	Stack traces con datos sensibles	Errores genéricos en producción
I-03	Info Disclosure	Logs con tokens o PII	Mask de campos sensibles antes de loguear
I-04	Info Disclosure	Enumeración de usuarios por login	Mensajes genéricos "credenciales inválidas"
I-05	Info Disclosure	Listado de endpoints en producción	Docs OpenAPI sólo con auth de admin
D-01	DoS	Flood de requests sin rate limit	Nginx + slowapi (futuro)
D-02	DoS	Queries lentas (N+1)	Eager loading + indexes
D-03	DoS	Pool de conexiones agotado	Pool dimensionado + timeouts
D-04	DoS	Memory exhaustion en exports	Streaming responses + límites
E-01	Elev. Privilege	Endpoint sin require_role	Tests automatizados de RBAC
E-02	Elev. Privilege	Escalamiento via cambio de rol propio	Endpoint /auth/role sólo admin
E-03	Elev. Privilege	Confused deputy via param tampering	Validación de ownership en cada acción

ID	Categoría	Amenaza	Mitigación
E-04	Elev. Priv- ilege	Bypass por bug en lógica de filtros	Pruebas integradas multi- tenant

5.4. Árboles de ataque

5.4.1. Árbol 1: Brecha cross-tenant

- **Objetivo:** ver datos de otra empresa
 - Modificar parámetro `empresa_id` en URL
 - Mitigación: el filtro se aplica server-side basado en el JWT, no en el path
 - Forzar rol auditor (acceso global)
 - Endpoint PUT `/auth/role` sólo admin + log de cambios
 - Ataque a la integración n8n (canal de salida)
 - Webhooks autenticados + IP allowlist

5.4.2. Árbol 2: Falsificar métricas de cumplimiento

- **Objetivo:** aparentar compliance sin haberlo logrado
 - Modificar status del control vía API directa → requiere rol auditor/admin
 - Editar snapshots históricos → tabla append-only, no update
 - Manipular detección de sector para evitar controles sectoriales → validación en backend

5.4.3. Árbol 3: Tomar control admin

- **Objetivo:** obtener cuenta admin
 - Phishing al admin → MFA + capacitación (A.6.3)
 - Bug en `require_role` → tests RBAC end-to-end
 - Comprometer Supabase → fuera del scope; depende del SLA del proveedor (R-15)
 - Reutilizar token de admin desconectado → rotación + logout invalidante

5.5. Hoja de ruta de mitigaciones

6. Trazabilidad Requisitos → Diseño

7. Conclusiones

La Etapa 2 consolida un diseño que responde a los requisitos formulados en la Etapa 1 y se sostiene sobre tres pilares:

1. **Arquitectura por capas** con separación estricta de responsabilidades, justificada por requisitos de modificabilidad y mantenibilidad.

Plazo	Mitigaciones	Prioridad
Inmediato (<2 semanas)	<code>_empresa_filter</code> en todos los routers, CSP headers, tests RBAC, mask de logs, errores genéricos en prod	Crítica
Corto plazo (1-3 meses)	MFA opcional, rate limiting (slowapi), tabla <code>audit_log</code> , secret rotation, escaneo de dependencias (CI)	Alta
Mediano plazo (3-6 meses)	WAF, monitoreo de anomalías (SIEM ligero), backups automáticos validados, simulacros de incidente	Media

Req. Etapa 1	Elemento de diseño	Justificación
RNF-Seg-01 (Confidencialidad)	TLS 1.3 + JWT + filtro <code>empresa_id</code>	Cumple CIA-C; mitiga R-01, I-01
RNF-Seg-02 (Integridad)	Bitácora + ORM parametrizado	Mitiga T-01, T-02, R-01
RNF-Seg-03 (Disponibilidad)	Pool + healthcheck + backups	Mitiga D-01..D-04
RNF-Esc	Async + Pool + indexes	Soporta carga horizontal
RNF-Mod	Routers + servicios + schemas	Permite extracción a microservicios
RF-Auth	Supabase Auth + RBAC factory	Cumple historia HU-01
RF-Multi-tenant	<code>empresa_id</code> FK + filtros	Cumple HU-12
RF-Reportes	Endpoint <code>/dashboard/export</code> + n8n	Cumple HU-08

2. **Análisis de riesgos cuantificado** (15 riesgos, 1 crítico, 7 altos) con estrategia de tratamiento alineada a ISO 27005 y mapeada al Anexo A de ISO 27001:2022.

3. **Modelado de amenazas híbrido PASTA + STRIDE** con 24 amenazas catalogadas, 3 árboles de ataque y hoja de ruta de mitigaciones priorizada.

El diseño no es estático: cada hallazgo de las etapas siguientes (codificación, pruebas, despliegue) puede generar un nuevo riesgo que se incorporará a esta matriz. El mantenimiento continuo del modelo de amenazas forma parte del ciclo PDCA exigido por ISO 27001:2022.

Referencias

- ISO/IEC 27001:2022 — Information security management systems — Requirements.
- ISO/IEC 27005:2022 — Guidance on managing information security risks.
- OWASP Application Security Verification Standard (ASVS) 4.0.3.
- OWASP Top 10:2021.
- Shostack, A. (2014). *Threat Modeling: Designing for Security*.

- UcedaVélez, T., & Morana, M. M. (2015). *Risk Centric Threat Modeling: Process for Attack Simulation and Threat Analysis (PASTA)*.